Gemini Chatbot (TASK 1)

Overview

Build a minimal web-based chatbot that uses Google's **Gemini API**.
The chatbot should support text conversation, document upload, image upload, and basic chat context. Users must be able to converse naturally, and reset the conversation easily.

The goal is to demonstrate API integration, file handling, and simple state management , not advanced AI/ML techniques.

Core Features

1. Chat interface

Provide a minimal chat UI where users can:

- Type messages
- View bot responses
- See their chat history for the current session

The UI must include:

- A text message input
- A **Send** button
- A **Document Upload** button (PDF/TXT)
- An **Image Upload** button (PNG/JPG)
- A **New Chat** button to reset context

Styling can be basic , functionality is the priority.

2. Document support

Support uploading **PDF** and **TXT** files.

3. Image support

Support uploading **PNG/JPG** images.

Send the image to the Gemini model alongside the user's message.

4. Basic chat context

Track conversation history only for the **current chat**.

Context should include:

- All user + bot messages
- Uploaded document text
- Uploaded image (optional)

When sending a new message:

- Pass recent messages + any uploaded file data to the Gemini API
- Bot responses should consider prior messages

Context should reset only when:

- User clicks **New Chat**

Context does not persist across sessions or server restarts.

5. New chat / reset

Support starting a completely fresh chat session.

On **New Chat**:

- Clear message history
- Clear document text
- Clear uploaded image
- Start a new chatId

The new chat must not have access to previous chat context or uploads.

Example Scenario

Example 1: Document Q&A

User uploads `notes.pdf`.
Then asks:

- “Summarize the document.”

Expected:
Bot generates a summary based on extracted PDF text.

User then asks:

- “What was the third point mentioned?”

Expected:
Bot answers using document context + previous conversation.

Example 2: Image Q&A

User uploads `photo.jpg`.
Then asks:

- “What’s in the image?”

Expected:
Bot describes objects in the uploaded image.

User asks:

- “Is the person smiling?”

Expected:
Bot answers using the same uploaded image.

Example 3: Context Reset

User: “What did I upload earlier?”
Bot: “You uploaded a document discussing XYZ.”

User clicks **New Chat**.

User: “What did I upload earlier?”

Expected:
Bot: “No files have been uploaded yet.” (fresh context)

Constraints

- Backend required (Node.js or Python).
- Frontend can be any simple framework (React recommended).
- Store chat state **in memory** only.
- No database, deployment, sessions, or authentication required.

- Only basic PDF/TXT extraction required.
- Only PNG/JPG for image uploads.
- Avoid complex RAG, embeddings, or chunking.

The solution must remain responsive and simple to run.

Bonus Challenges (Optional)

- Display uploaded image preview.
- Allow multiple chats to be listed in the UI.
- Add loading indicators for file uploads and responses.

Deliverables

Primary deliverable

A GitHub repository containing:

- Frontend source code
- Backend source code
- `README.md` explaining:
    - How to install
    - How to set Gemini API key
    - How to run frontend + backend
    - Example usage steps

Optional deliverable

A deployed version on Netlify/Vercel (frontend) + Render(or any alternative ; for backend).